

Question 1 – Zomato Food Ordering Create a class Restaurant to simulate Zomato's restaurant system. Requirements:

1. The constructor should accept restaurant_name and menu (dictionary of items → price).
2. Define a method showMenu() that displays all items with prices. o Example: Pizza - ₹200, Burger - ₹120
3. Define a method orderFood(item, qty) that: o Checks if the item is available in the menu. o If available → prints the bill (price * qty). o If not available → prints "Item not available".
4. Create two restaurant objects (Dominos, KFC) with different menus. 5. Place orders from both restaurants.

class zamato:

```
def __init__(self,i,j):
```

```
    self.name=i
```

```
    self.menu=j
```

```
def showmenu(self):
```

```
    print("restuarent name:",self.name)
```

```
    print("resuarent menu",self.menu)
```

```
def orderfood(self,item,qty):
```

```
    self.item=item
```

```
    self.qty=qty
```

```
    # print(self.item)
```

```
    # print(self.qty)
```

```
    print("-----ordered food-----")
```

```
    if self.item in self.menu:
```

```
        price=self.menu[self.item]
```

```
        print(self.item,"is available in",self.name,"total bill is",price*qty)
```

```
    else:
```

```
        print(self.item," is not available","in",self.name)
```

```
# abc=zamato("pizza_palace",{"Pizza": 250, "Burger": 150})
```

```
# abc.showmenu()
```

```
# abc.orderfood(item="pizza",qty=2)
dominos=zamato("dominos",{"Pizza": 250, "Burger": 150})
kfc=zamato("kfc",{"Pizza": 250, "Burger": 150,"fries":199})
dominos.showmenu()
kfc.showmenu()
dominos.orderfood(item="Pizza",qty=2)
kfc.orderfood(item="biryani",qty=4)
```

o/p:

restuarent name: dominos

resuarent menu {'Pizza': 250, 'Burger': 150}

restuarent name: kfc

resuarent menu {'Pizza': 250, 'Burger': 150, 'fries': 199}

-----ordered food-----

Pizza is available in dominos total bill is 500

-----ordered food-----

biryani is not available in kfc

2. uestion 2 – Uber Ride Booking Create a class Driver to simulate Uber driver details. Requirements:

1. The constructor should accept driver_name, car_model, and per_km_rate.
2. Define a method showDriver() that prints driver details. o Example: Driver: Raj, Car: Swift, Rate: ₹20/km
3. Define a method calculateFare(distance) that calculates and prints the fare. o Example: Distance: 10 km, Fare: ₹200
4. Create 3 driver objects with different details.
5. For each driver, calculate fare for a trip (e.g., 8 km, 12 km, 15 km).

class driver:

```
def __init__(self,driver_name,car_model,per_km_rate):
    self.driver_name=driver_name
    self.car_model=car_model
```

```
        self.per_km_rate=per_km_rate
def showdriver(self):
    print("-----Driver Details-----")
    print(f" driver name:--{self.driver_name}")
    print(f" car:--{self.car_model}")
    print(f" rate:-- ₹{self.per_km_rate}/km")
def claculatefare(self,distance):
    self.distance=distance
    fare=self.distance*self.per_km_rate
    print(f"total fare:---₹{fare}")
driver1=driver("vikram","Swif",20)
driver2=driver("iran","thar",50)
driver3=driver("nandu","BMW",60)
driver1.showdriver()
driver2.showdriver()
driver3.showdriver()
driver1.claculatefare(8)
driver2.claculatefare(12)
driver3.claculatefare(15)
```

o/p:

-----Driver Details-----

driver name:--vikram

car:--Swif

rate:-- ₹20/km

-----Driver Details-----

driver name:--iran

car:--thar

rate:-- ₹50/km

-----Driver Details-----

driver name:--nandu

car:--BMW

rate:-- ₹60/km

total fare:---₹160

total fare:---₹600

total fare:---₹900

Question 3 – Flipkart Shopping Cart Create a class Product to simulate shopping on Flipkart. Requirements:

1. The constructor should accept product_name, price, and stock.
2. Define a method showDetails() that displays product details. o Example: Laptop - ₹50000, Stock: 10
3. Define a method buyProduct(qty) that: o If enough stock → reduces stock and prints total cost. o If not enough stock → print "Out of Stock".
4. Create 3 product objects (Laptop, Phone, Shoes) with different stock and price.
5. Simulate a shopping cart by: o Buying 2 items from Laptop. o Buying 1 item from Phone. o Trying to buy more Shoes than available stock.

class product:

```
print("-----product details-----")

def __init__(self,product_name,price,stock):

    self.product_name=product_name

    self.price=price

    self.stock=stock


def showdetails(self):

    print(f"product name:---{self.product_name}")

    print(f"price:---₹{self.price}")

    print(f"stock:---{self.stock}")

def buyProduct(self,qty):

    self.qty=qty

    if self.stock>=qty:
```

```
total=self.qty*self.price
self.stock-=qty
print("total cost:--",total,"remaing stock",self.stock)
else:
    print("out of stock || available stock:---",self.stock)

product1=product("laptop",80000,10)
product2=product("phone",30000,25)
product3=product("shoes",4000,15)
product1.showdetails()
product2.showdetails()
product3.showdetails()
product1.buyProduct(2)
product2.buyProduct(1)
product3.buyProduct(20)

o/p:
-----product details-----
product name:---laptop
price:---₹80000
stock:---10
product name:---phone
price:---₹30000
stock:---25
product name:---shoes
price:---₹4000
stock:---15

total cost:-- 160000 remaing stock 8
total cost:-- 30000 remaing stock 24
out of stock || available stock:--- 15
```

